Loop App · Arquitetura e design do backend + plano de implementação

Versão 0.1 · 2026-11-10 · Status: **proposta para aprovação** (nenhum código escrito) Base: `ANALISE_PRD_FRD.md`, `PERGUNTAS_ABERTAS.md`, `apps/orchestrator/CLAUDE.md` Método: `backend-design:design` (6 passos do *think-before-coding*) + pesquisa web.

Evidência: **[A]** declarado · **[B]** fonte externa · **[C]** hipótese · **[D]** desconhecido.

---

0. Resumo executivo

O backend tem **quatro tipos de componente** que compartilham um núcleo de domínio:

#	Componente	Tipo	Onde roda
C1	API de comandos da plataforma	HTTP (RPC por operação)	`apps/web` (Next.js route handlers, OpenNext em Workers)
C2	Receptor de webhooks do GitHub	Webhook receiver	`apps/worker` (Worker dedicado)
C3	Consumidores de fila (sync com repo, e-mail, WhatsApp, agentes)	Queue consumer + integração externa	`apps/worker`
C4	Tarefas agendadas (reconciliação, retenção, backup lógico)	Cron	`apps/worker`

Núcleo compartilhado: `packages/core` (operações de domínio), `packages/db` (schema + acesso com escopo de tenant), `packages/model-schema`, `packages/sim-engine`.

Decisões propostas (precisam do seu OK):

ID	Decisão	Motivo curto
ADR-01	**Um banco D1 com `tenant_id` em toda tabela**, não um banco por tenant	≤ 20 tenants [C]; uma migração só; D1 suporta até 10 GB por banco no plano pago [B]
ADR-02	**Drizzle ORM** sobre D1	Better Auth não tem adapter D1 nativo; o caminho indicado é Drizzle com provider sqlite [B]
ADR-03	**Better Auth + plugin organization** (organização = tenant), com papéis customizados	Convites, membros e papéis prontos [B]; roda no mesmo Worker
ADR-04	**Worker separado** (`apps/worker`) para webhook, filas e cron	Next/OpenNext serve UI e API; filas e cron ficam fora do ciclo de request
ADR-05	**Outbox transacional** para todo efeito externo (PR, issue, e-mail, WhatsApp)	Nenhum efeito externo dentro da transação; reenvio seguro
ADR-06	**Operações nomeadas por verbo** (`PublishModelVersion`, `ApproveProposal`), não CRUD genérico	O domínio é fluxo de trabalho com estados
ADR-07	**Simulação no navegador**; o servidor só grava `seed + parâmetros + resultado` e pode reexecutar para auditoria	Reaproveita o DIAGRAMA_V2; custo zero no servidor
ADR-08	**Modelo versionado e imutável** (`ModelVersion`), concorrência otimista por número de versão	Histórico completo; sem perda silenciosa

1. Passo 1 · Contexto

- **Chamadores de C1:** navegador do desenvolvedor e do cliente (UI que repete a ação se falhar). Celular é caso principal para o cliente [A].
- **Chamador de C2:** GitHub (repete a entrega em caso de falha; pode entregar fora de ordem).
- **C3:** filas Cloudflare Queues; entrega com novas tentativas configuráveis (padrão 3) e fila de mensagens mortas (DLQ) opcional [B]. Tratar como **pelo menos uma vez** (não confirmei "exatamente uma vez") [C].
- **C4:** Cron Triggers do Workers.
- **Volume esperado [C]:** ≤ 50 usuários ativos, ≤ 10 projetos ativos, dezenas de requisições por minuto em pico, < 1.000 mensagens de fila por dia.
- **Latência:** página < 2 s p95; comando de escrita < 300 ms p95 (sem chamada externa no caminho).

2. Passo 2 · Modelo de dados

2.1 Entidades e invariantes

Todas as tabelas de negócio têm `id` (ULID, texto), `tenant_id`, `created_at`, `updated_at`. Datas em UTC ISO-8601.

Tabela	Campos principais	Invariantes (garantidas no banco quando possível)
`organization` (Better Auth)	id, slug, name	`slug` único; slug = rota `/[tenant]`
`member` (Better Auth)	organization_id, user_id, role	único (organization_id, user_id); `role` ∈ papéis do §4
`project`	tenant_id, name, template_id, phase, repo_full_name, repo_installation_id, sync_state	`phase` ∈ {draft, diagram, studio, proposal, delivery, closed}; `sync_state` ∈ {unlinked, linked, divergent, error}
`form`	tenant_id, project_id, schema_json, status	`status` ∈ {draft, open, closed}; schema válido contra JSON Schema 2020-12
`form_response`	tenant_id, form_id, user_id, data_json, submitted_at, draft	único (form_id, user_id) para rascunho; envio cria linha imutável
`model`	tenant_id, project_id, kind, purpose, current_version	`kind` ∈ {flow_map, causal_loop, influence, archetype, vsm, rich_picture, activity_model, iceberg}; `purpose` ∈ {system, change}
`model_version`	tenant_id, model_id, version, data_json, boundary_json, created_by, hash	único (model_id, version); **imutável** após criada; `data_json` válido contra `model-schema`
`parameter`	tenant_id, model_version_id, key, value, unit, evidence, source_note	único (model_version_id, key); `evidence` ∈ {A,B,C,D} **NOT NULL**
`scenario`	tenant_id, model_id, name, overrides_json, created_by	overrides só referenciam chaves existentes
`simulation_run`	tenant_id, scenario_id, model_version_id, seed, horizon_days, result_json, engine_version	imutável; (seed, model_version, overrides, engine_version) reproduz o resultado

Tabela	Campos principais	Invariantes (garantidas no banco quando possível)
`intervention`	tenant_id, project_id, target_ref, description, human_impact_json, registry_refs, status, approved_by, approved_at	`status` ∈ {proposed, approved, rejected, published, done}; agente com acesso a dados exige `approved_by` de `client_admin`
`comment`	tenant_id, project_id, target_ref, user_id, area, stance, body, parent_id	`area` ∈ {private, shared}; `stance` ∈ {note, question, disagree, agree}
`proposal`	tenant_id, project_id, version, content_json, status, approved_by, approved_at	`status` ∈ {draft, sent, approved, superseded}; uma `approved` por projeto por vez; aprovada é imutável
`outbox`	tenant_id, kind, payload_json, dedupe_key, status, attempts, next_attempt_at, last_error	único (dedupe_key); `status` ∈ {pending, sent, failed, dead}
`sync_event`	tenant_id, project_id, direction, artifact, ref, pr_url, status	trilha de auditoria do sync
`webhook_delivery`	delivery_id, event, received_at, processed_at, status	`delivery_id` (X-GitHub-Delivery) único → deduplicação
`idempotency_key`	tenant_id, user_id, key, op, response_json, expires_at	único (tenant_id, user_id, key)
`audit_log`	tenant_id, user_id, action, target_ref, ip_hash, at	só inserção

Índices: toda FK indexada; índices compostos começando por `tenant_id` nas consultas de lista (`(tenant_id, project_id, created_at)`).

2.2 Ciclos de vida (máquinas de estado)

```
Project:   draft → diagram → studio → proposal → delivery → closed
                          ▲_____________________| (volta permitida só por dev)

Proposal:  draft → sent → approved → superseded
              └──────→ draft (edição gera nova versão; 'sent' nunca é editada)

Intervention: proposed → approved → published (PR/issue aberta) → done (issue fechada)
                     └→ rejected

Outbox:    pending → sent
              └→ (falha) pending com backoff → failed → dead (após N tentativas; alerta)
```

Transições são **funções do domínio** (`packages/core`), nunca `UPDATE status` solto na rota.

2.3 Retenção e exclusão

- Prazo de guarda de mensagens/documentos: **A DEFINIR** (pergunta SI-05). Até lá: sem exclusão automática.
- Exclusão de tenant: soft-delete com prazo (30 dias [C]) e depois exclusão física por cron, com registro em `audit_log`.
- LGPD: dados pessoais mínimos (nome, e-mail); base legal e encarregado dependem de SI-10/SI-11.

---

3. Passo 3 · Modos de falha

Componente	Falha	Efeito	Tratamento
C1	Duplo clique em "aprovar proposta"	Duas aprovações	`Idempotency-Key` + transição de estado condicional (`WHERE status='sent'`)
C1	Dois devs publicam versão do modelo ao mesmo tempo	Sobrescrita	Concorrência otimista: `expected_version`; conflito → 409
C1	`data_json` inválido	Modelo corrompido	Validação com `model-schema` antes de gravar → 422 com caminho do erro
C1	Usuário sem vínculo acessa outro tenant	Vazamento	Escopo no acesso a dados (§4) → 404; teste em CI
C1	D1 indisponível ou lento	Erro na UI	Timeout curto, 503 com `retry-after`; UI preserva formulário
C2	Assinatura inválida	Evento forjado	Verificar `X-Hub-Signature-256` (HMAC-SHA256, comparação em tempo constante) → 401 [B]
C2	Mesma entrega repetida	Processamento duplo	Deduplicar por `X-GitHub-Delivery` em `webhook_delivery` [B]
C2	Processamento lento	GitHub marca falha	Responder 202 rápido; trabalho vai para fila
C2	Eventos fora de ordem	Estado errado	Estado derivado do evento mais recente por `updated_at` do recurso; reconciliação no cron
C3	GitHub API 5xx/rate limit	Sync atrasa	Retry com backoff exponencial; respeitar `retry-after`; DLQ após limite [B]
C3	Arquivo no repo mudou desde o último sync	Sobrescrever trabalho do cliente	Comparar SHA do blob; se diferente → `sync_state=divergent`, abrir PR de diff, não sobrescrever
C3	E-mail/WhatsApp rejeitado	Cliente não avisado	Marcar `failed`; mostrar na UI; sem retry infinito
C3	Mensagem processada 2× (entrega pelo menos uma vez)	PR duplicado	`dedupe_key` na outbox + verificação "já existe PR para este ref?"
C4	Cron roda 2× ou atrasa	Trabalho duplicado	Cron é idempotente (seleciona por estado, não por horário)
Todos	Segredo vazado em log	Incidente	Lista de campos redigidos no logger; secret scan no CI
Todos	Restauração necessária	Perda de dados	D1 Time Travel (qualquer minuto dos últimos 30 dias no plano pago; restauração destrói o estado atual) [B] + export lógico diário para R2

4. Passo 4 · Autorização

- **Principal:** usuário autenticado (sessão Better Auth) ou instalação do GitHub App (C2/C3) ou o próprio sistema (C4).
- **Escopo:** tenant (organização) + área (private/shared/studio/proposal) + papel.
- **Papéis** (custom roles no plugin organization):

Papel	Escopo
`owner_dev`	tudo no tenant; gerencia membros; MFA obrigatório
`dev`	áreas privada e compartilhada; modelos; intervenções; MFA obrigatório
`client_admin`	área compartilhada; aprova proposta e intervenções; convida membros do cliente
`client_member`	área compartilhada; formulários; comentários; roda cenários
`viewer`	leitura de compartilhada, studio e proposta

- **Ponto de aplicação (três camadas, todas obrigatórias):**
  1. **Middleware** resolve `tenant` pelo slug e carrega `member`; sem vínculo → 404 (não 403, para não revelar existência).
  2. **Operação de domínio** verifica a permissão da ação (`can(member, 'proposal.approve')`).
  3. **Acesso a dados** só por `tenantDb(ctx)`, que injeta `tenant_id` em toda consulta. Consulta direta ao driver fora desse módulo é proibida (regra de lint).
- **Área privada:** consultas de papéis de cliente filtram `area='shared'` na camada 3, não na UI.
- **GitHub App:** permissão mínima — `contents: write` e `pull_requests: write` só nos repos instalados, `issues: write`, `metadata: read`. Token de instalação de curta duração gerado por request; chave privada como secret do Worker.
- **Webhook:** só aceita evento com assinatura válida e `installation.id` ligado a um `project`.

---

5. Passo 5 · Idempotência e concorrência

Operação	Repetição (replay)	Concorrência	Fora de ordem
`ApproveProposal`	`Idempotency-Key` devolve a mesma resposta	`UPDATE … WHERE status='sent' AND version=?`	n/a
`PublishModelVersion`	mesma key → mesma versão	`expected_version` → 409	n/a
`RecordSimulationRun`	key = hash(seed, versão, overrides, engine) → mesma linha	inserção única	n/a
`SubmitFormResponse`	key do cliente; envio cria linha imutável	rascunho por (form, user) com `updated_at`	n/a
Sync → PR (C3)	`dedupe_key = project:artifact:model_version`	uma mensagem por projeto por vez (lock por `project.sync_state`)	versão maior vence; menor é descartada
Webhook (C2)	`delivery_id` único	inserção única	compara `updated_at` do recurso
Cron (C4)	seleção por estado	`UPDATE … WHERE status='pending'` como reserva	n/a

Outbox (ADR-05): a transação de domínio grava a mudança **e** a linha em `outbox`. Um produtor (no fim do request e no cron de 1 min) envia linhas `pending` para a fila. O consumidor marca `sent` só após sucesso externo. Nenhuma chamada externa acontece dentro de transação D1.

---

6. Passo 6 · Observabilidade

- **Logs estruturados (JSON)** em todo componente: `request_id`, `tenant_id`, `user_id` (hash), `op`, `status`, `duration_ms`. Nunca: tokens, corpo de formulário, e-mails completos.
- **Métricas (RED por operação):** taxa, erros, duração. Extras: outbox `pending` por idade, mensagens na DLQ, falhas de sync, entregas de webhook rejeitadas, custo de IA/WhatsApp por projeto (Fase 3).
- **Alertas:** DLQ > 0; outbox `pending` > 15 min; falha de sync > 1 h; taxa de 5xx > 2% em 5 min; webhook com assinatura inválida > 10/h.
- **Healthcheck:** `GET /api/health` (C1) verifica D1 com `SELECT 1`; o Worker C2–C4 expõe `/health` com idade da última execução do cron.
- **Trace:** `request_id` propagado do request para a linha da outbox e para a mensagem da fila.

---

7. Blueprint

7.1 Estrutura do monorepo

```
apps/
  web/            Next.js (UI + API de comandos C1) · OpenNext · binding D1, R2, Queue
(produtor)
  worker/         Worker: fetch (webhook C2, /health), queue (C3), scheduled (C4)
  orchestrator/   registry (existente)
packages/
  core/           operações de domínio, máquinas de estado, permissões (sem I/O direto)
  db/             schema Drizzle, migrações, tenantDb(ctx), repositórios
  model-schema/   JSON Schema + tipos de Model, Station, Flow, Parameter, Scenario
  sim-engine/     simulação determinística (extraída do DIAGRAMA_V2)
  integrations/   github (App, PRs, issues), email, whatsapp — clientes finos com retry
```

7.2 Operações (C1) — contrato

Todas: `POST /api/t/{tenant}/ops/{operation}`, corpo JSON validado, header `Idempotency-Key` obrigatório nas mutações, resposta `{ data }` ou `{ error: { code, message, details } }`. Leituras usam `GET` com recursos.

Operação	Quem pode	Efeito	Erros
`OpenProject`	owner_dev, dev	cria projeto a partir de template (forms + modelo vazio)	409 nome duplicado
`LinkRepository`	owner_dev	liga instalação do GitHub App ao projeto	422 instalação inválida
`PublishModelVersion`	dev	nova `model_version` + parâmetros	409 versão, 422 schema
`CreateScenario`	dev, client_*	cenário com overrides	422 chave inexistente
`RecordSimulationRun`	todos com acesso ao studio	grava run reprodutível	422 engine incompatível
`SaveFormDraft` / `SubmitFormResponse`	client_*, dev	rascunho / envio imutável; atualiza `parameter` com evidência A	422 schema
`PostComment`	conforme área	comentário; pergunta ao cliente gera e-mail (outbox)	403 área

Operação	Quem pode	Efeito	Erros
`ProposeIntervention`	dev	intervenção `proposed`	422
`DecideIntervention`	client_admin	approved/rejected	409 estado
`SendProposal`	owner_dev	proposta `sent` (congela versão)	409 estado
`ApproveProposal`	client_admin	`approved` + outbox de publicação no repo	409 estado
`InviteMember` / `ChangeRole`	owner_dev, client_admin (só papéis do cliente)	convite (Better Auth)	403

Leituras: `GET /api/t/{tenant}/projects`, `/projects/{id}`, `/models/{id}?version=`, `/scenarios/{id}/runs`, `/comments?target=`, `/proposals/{id}`, com paginação por cursor.

7.3 Worker (C2–C4)

```
fetch:
  POST /webhooks/github   verifica assinatura → grava webhook_delivery (único) → enfileira →
202
  GET  /health
queue "loop-sync":       publica artefatos no repo (PR) · lê estado de PR/issue
queue "loop-notify":     e-mail (depois WhatsApp)
queue "loop-dlq":        consumidor só registra e alerta
scheduled:
  */1 * * * *   drena outbox pendente para as filas
  */15 * * * *  reconcilia estado de PRs/issues com GitHub
  17 3 * * *    export lógico do D1 para R2; limpa idempotency_key expiradas; exclusões
vencidas
```

Configuração das filas: `max_retries` 5, backoff exponencial, `dead_letter_queue = loop-dlq` (padrão do Cloudflare é 3 tentativas e descarte sem DLQ — por isso configurar explicitamente) [B].

7.4 Sync com o repositório do cliente

1. `ApproveProposal` ou `PublishModelVersion` (quando marcado "publicar") grava outbox `sync.publish`.
2. Consumidor obtém token de instalação, lê o SHA atual dos arquivos `loop/model/*.json` e `loop/decisions/*`.
3. Se o SHA difere do último `sync_event` → `divergent`; abre PR somente com o diff para revisão; para.
4. Senão cria branch `loop/sync-<versão>`, faz commit dos artefatos validados, abre PR, grava `sync_event` e `pr_url`.
5. Webhook `pull_request.closed(merged)` → `sync_event.status=merged`; issues com label `loop` → atualizam `intervention`.

7.5 Segurança (piso)

Segredos só em `wrangler secret` (chave do GitHub App, segredo do webhook, chave de e-mail); nada no repositório. MFA para devs. CSRF: cookies `SameSite=Lax` + verificação de origem nas mutações. Limite de taxa por usuário nas operações de escrita e por IP no login. Cabeçalhos de segurança (CSP restrita) no app. Uploads em R2 com tipo validado por conteúdo e URL assinada de curta duração.

---

8. Trade-offs

Escolha	Alternativa	Ganho	Custo / quando revisitar
D1 único com `tenant_id`	D1 por tenant	Uma migração, consultas cruzadas para a Loop	Isolamento lógico, não físico; revisitar se um cliente exigir isolamento físico ou passar de ~2 GB
D1	Prisma Postgres	Mesmo deploy, custo zero, Time Travel	Sem pgvector/RAG; revisitar na Fase 3 (E_5, E_10)
Worker separado	Tudo no OpenNext	Filas e cron fora do request	Dois deploys; mitigado por `packages/` compartilhados
Outbox	Chamada externa direta	Sem PR/e-mail fantasma ou perdido	Latência de até 1 min no efeito; aceitável
RPC por operação	REST CRUD	Regras de estado explícitas	Menos "automático"; mais claro
Simulação no cliente	No servidor	Instantânea, sem custo	Resultado gravado pode ser forjado → servidor reexecuta por amostragem para auditoria

9. Plano de implementação

Cada etapa termina com o portão G5/G6 do `CLAUDE.md` (testes + build + verificação). Skills do STACK indicadas por etapa.

W1 · Fundação (bloqueantes: TD-01, TD-02, TD-03, TD-04, TD-09, SI-09, SI-10, CM-01, CM-07)

Passo	Entrega	Verificação	Skills
1.1	`packages/db`: schema Drizzle (§2), migração inicial, `tenantDb(ctx)`	Teste: consulta sem tenant falha; lint proíbe driver direto	data-modeling-discipline, migration-safety
1.2	Better Auth + organization + papéis customizados + MFA para dev	Teste de login, convite e papel	auth-and-authorization
1.3	Middleware de tenant + `can()` em `packages/core`	**Teste de isolamento** entre dois tenants (Gherkin FR-AUTH)	security-discipline
1.4	`Idempotency-Key` + formato de erro + logger com redação	Teste de replay devolve mesma resposta	idempotency-and-side-effects, error-handling-as-design
1.5	`apps/worker` com `/health`, cron vazio, filas + DLQ configuradas	Deploy de staging; health 200	observability-by-default
1.6	GitHub App (permissão mínima) + `LinkRepository` + webhook com assinatura e dedupe	Teste com payload assinado e repetido	security-discipline
1.7	CI: testes, secret scan, migração aplicada em banco de teste; backup lógico + restauração ensaiada	Registro da restauração	testing-with-discernment

Saída W1: isolamento verde, tenant CleanSea ligado ao repo, deploy em staging, restauração ensaiada.

W2 · Loop Diagrama (bloqueantes: OP-01…OP-16 marcados, RG-01, RG-02)

Passo	Entrega	Verificação	Skills
2.1	`packages/model-schema` (flow_map primeiro)	Mapa CleanSea do DIAGRAMA_V2 validado como dados	data-modeling-discipline
2.2	`packages/sim-engine` extraído; `engine_version`	Teste de reprodutibilidade (mesmo seed = mesmo resultado); cenário "anuência lenta" marca P	testing-with-discernment
2.3	`PublishModelVersion`, `CreateScenario`, `RecordSimulationRun`	Concorrência otimista (409) testada	query-discipline
2.4	Formulários: `SaveFormDraft`, `SubmitFormResponse` → `parameter` evidência A	Resposta atualiza parâmetro com origem	—
2.5	Comentários + outbox de e-mail + consumidor `loop-notify`	E-mail enviado uma vez mesmo com retry	idempotency-and-side-effects

Saída W2: mapa do porto-piloto a partir de dados, ≥ 60% de parâmetros A/B, cliente rodou cenário.

W3 · Proposta, sync e operação (bloqueantes do bloco W3)

Passo	Entrega	Verificação	Skills
3.1	Intervenções + `DecideIntervention` (aprovação de `client_admin` para agentes com acesso a dados)	Teste: sem aprovação não publica	auth-and-authorization
3.2	Proposta: `SendProposal`, `ApproveProposal` (imutável após aprovada)	Duplo clique = uma aprovação	idempotency-and-side-effects
3.3	Sync com repo (§7.4), incluindo `divergent`	Teste com SHA divergente não sobrescreve	error-handling-as-design
3.4	Reconciliação no cron + webhooks de PR/issue	Estado converge após evento perdido	—
3.5	Métricas, alertas, painel de custo; WhatsApp (após SI-03/SI-04)	Alertas disparam em teste; orçamento respeitado	observability-by-default, performance-and-scaling
3.6	Revisão de segurança e carga leve antes da produção	Relatório sem achado crítico	security-reviewer, incident-thinker

Saída W3 (produção): proposta aprovada na plataforma, PRs/issues no repo da CleanSea, alertas ativos, medição de linha de base registrada.

Estimativas de prazo: **A DEFINIR** após CM-01 (escopo) e capacidade da equipe.

---

10. O que revisitar com o crescimento

1. D1 → Postgres (Prisma) quando entrar RAG/pgvector ou analytics pesado.
2. Réplicas de leitura do D1 (exigem a Sessions API) se a latência fora do Brasil importar [B].
3. Durable Objects para colaboração em tempo real em workshops.
4. Banco por tenant se algum cliente exigir isolamento físico.
5. Motor de simulação no servidor para lote/otimização de cenários.

11. Perguntas de decisão para você

1. Aprova ADR-01 a ADR-08? (Principalmente: D1 único com `tenant_id` e Worker separado.)
2. O cliente usa o app principalmente no celular, com rede instável? Se sim, priorizo salvar rascunho offline nos formulários (passo 2.4).
3. A publicação no repo do cliente deve ocorrer **só após aprovação da proposta** ou também a cada versão do modelo marcada pelo dev? (Afeta 7.4.)

Fontes externas

- Cloudflare Queues — batching, retries e delays
- Cloudflare Queues — dead letter queues
- Cloudflare Queues — configuração
- Cloudflare D1 — limites
- Cloudflare D1 — Time Travel
- Cloudflare D1 — read replication
- Cloudflare D1 — FAQ
- GitHub — validar entregas de webhook
- Svix — segurança de webhooks do GitHub
- Better Auth — guia D1/Drizzle (skill comunitária)
- Better Auth UI — organizations
- Neon — Better Auth organization plugin
- Organization plugin e sessão de organização ativa